Student Management System – Frontend Code With Explanation

This document explains the complete JavaScript frontend code used in the Student Management System. Each section includes the actual code followed by a clear explanation of what it does and why it is needed.

1. API Configuration

```
const API_URL = "http://localhost:3000/api/students";
```

This constant stores the backend API endpoint. All requests such as fetching, adding, updating, and deleting students use this URL. Changing the backend address requires updating only this line.

2. DOM Element Selection

```
const form = document.getElementById("studentForm");
const tableBody = document.getElementById("studentsTable");
const cancelBtn = document.getElementById("cancelBtn");

const fullName = document.getElementById("full_name");
const age = document.getElementById("age");
const address = document.getElementById("address");
const level = document.getElementById("level");
const phone = document.getElementById("phone_number");
const joinedDate = document.getElementById("joined_date");
const gradDate = document.getElementById("expected_graduation_date");
```

These statements select HTML elements so JavaScript can read user input, update the table, and control buttons. Centralizing element selection improves readability and maintenance.

3. Application State

```
let students = [];
let editId = null;
```

The students array holds the current list of students fetched from the backend. The editId variable tracks whether the form is in add mode or edit mode. When editId is null, a new student is added; otherwise, an existing student is updated.

4. Fetching Students (Read)

```
async function fetchStudents() {
  const res = await fetch(API_URL);
  const data = await res.json();
  students = Array.isArray(data) ? data : data.data;
  renderStudents(students);
}
```

This function retrieves all student records from the backend using a GET request. The backend is treated as the source of truth. Once data is received, the table is re-rendered.

5. Adding and Updating Students (Create & Update)

```
async function saveStudent(data) {
```

```
  const isEdit = Boolean(editId);
  const url = isEdit ? `${API_URL}/${editId}` : API_URL;

  await fetch(url, {
    method: isEdit ? "PUT" : "POST",
    headers: { "Content-Type": "application/json" },
    body: JSON.stringify(data),
  });

  fetchStudents();
}
```

This function handles both adding and editing students. POST is used to add new records, and PUT is used to update existing ones. After saving, data is re-fetched to keep the UI synchronized.

6. Deleting Students

```
async function deleteStudent(id) {
  await fetch(`${API_URL}/${id}`, { method: "DELETE" });
  fetchStudents();
}
```

This function removes a student record using a DELETE request. After deletion, the updated list is fetched again so the table updates instantly.

7. Rendering the Table

```
function renderStudents(list) {
  tableBody.innerHTML = "";
  list.forEach((s, i) => {
    tableBody.innerHTML += `<tr>
      <td>${i + 1}</td>
      <td>${s.full_name}</td>
      <td>${s.age}</td>
      <td>${s.level}</td>
    </tr>`;
  });
}
```

This function dynamically creates table rows based on student data. Each time data changes, the table is fully re-rendered to ensure accuracy.

8. Form Submission & Validation

```
form.addEventListener("submit", e => {
  e.preventDefault();
  saveStudent({
    full_name: fullName.value,
    age: Number(age.value),
    address: address.value,
    level: level.value,
    phone_number: phone.value,
    joined_date: joinedDate.value,
    expected_graduation_date: gradDate.value,
  });
});
```

Form submission is intercepted to prevent page reload. Validated data is sent to the backend for saving.

9. Application Initialization

```
fetchStudents();
```
This function call runs when the page loads. It ensures the table is populated with backend data immediately.